

Fall 2026 Exploratory Data Analysis with R

Lecture 11: Correlation vs. Causation

Brian Robertson
Federal Reserve Bank of Atlanta
2026-11-13

01

Recap & Agenda

01 Recap & Agenda

02 Why Correlation Isn't Causation

03 Spurious Correlation

04 Confounding Variables

05 Reverse Causation

06 Selection Bias

07 Simpson's Paradox

08 Checking Your Own Project

09 Recap

Quick Recap: Last Week

- **Code review** checks a function against the course's style conventions and common bug patterns — watch for `$` on a data frame, condition order in `case_when()`, and hidden single-input assumptions
- A **function** packages a block of code with a name, arguments, and a return value, so you write the logic once and call it as many times as you need
- **Functional programming** — `purrr::map()` and its typed variants apply a function to every element of a vector or list, instead of a copy-pasted call or a loop

Today: back to statistics, for one of the most important ideas in the course. Lecture 6 ended with “correlation does not imply causation.” Today we build a toolkit for figuring out *why* a correlation can mislead you — and how to check your own project for each of those reasons.

Goals for Today

- Recognize four specific reasons a correlation can mislead: spurious correlation, confounding variables, reverse causation, and selection bias
- Compare an overall correlation to correlations computed within subgroups, using `group_by()` and `summarise()` — never `aggregate()` or a loop
- Recognize **Simpson's paradox** — a relationship that reverses sign once you condition on a group variable
- Apply this toolkit to your own project before you submit your first full draft

No new homework or project update this week. The first full draft of your project is due today by 9am — that's a project milestone, not a numbered assignment from this lecture.

02

Why Correlation Isn't Causation

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation**
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap

Correlation Does Not Imply Causation

- Lecture 6: two variables can be strongly correlated without one causing the other.
- That sentence is true, but it is not very useful by itself — it does not tell you *why* a particular correlation might be misleading, or what to actually check before you trust one.

Today's goal: a short list of specific, checkable reasons a correlation can show up even when X has no direct causal effect on Y — so you have something concrete to look for, both in examples and in your own project.

Four Reasons X and Y Might Correlate Without X Causing Y

1. **Spurious correlation** — chance, or two variables that happen to share a trend
2. **Confounding** — a third variable drives both X and Y
3. **Reverse causation** — Y causes X , not the other way around
4. **Selection bias** — the sample itself, not the relationship, creates or hides the pattern

And one pattern that cuts across all four: **Simpson's paradox**, where a relationship flips direction once you split the data into groups. We'll come back to it after working through the four reasons above.

03

Spurious Correlation

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation**
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap

Spurious Correlation

A **spurious correlation** is a correlation with no real connection between the variables at all — not even through a third variable. It shows up for one of two reasons:

- **Pure chance** — with enough variables and enough comparisons, some pairs will show a strong correlation just by luck
- **A shared trend** — two variables that are both drifting up (or down) over time will look correlated with each other, even if neither one is driving the other

Neither reason involves any real link between X and Y .

A Genre of Examples

tylervigen.com/spurious-correlations — cited already in Lecture 6 — is built entirely on this idea: mine thousands of time series, and some pairs will correlate strongly by pure chance. Examples live on the site right now include:

- Per-capita **butter consumption** and the number of **epidemiologists in Colorado**
- Per-capita **margarine consumption** and the **divorce rate in Maine**

Each of these is a real, computed correlation — the website isn't making up the numbers. What's fake is any implied causal story. With enough variables measured over enough years, some pairs are *guaranteed* to line up by chance.

Spurious Correlation, Built From Scratch

A constructed illustration: two unrelated series, each just a random walk with no connection to the other.

```
set.seed(440)
n <- 72

trend_data <- tibble(
  month = 1:n,
  umbrellas_sold = cumsum(rnorm(n, mean = 0.5, sd = 2)),
  satellites_launched = cumsum(rnorm(n, mean = 0.4, sd = 2.5))
)

trend_data %>%
  summarise(cor = cor(umbrellas_sold, satellites_launched))
```

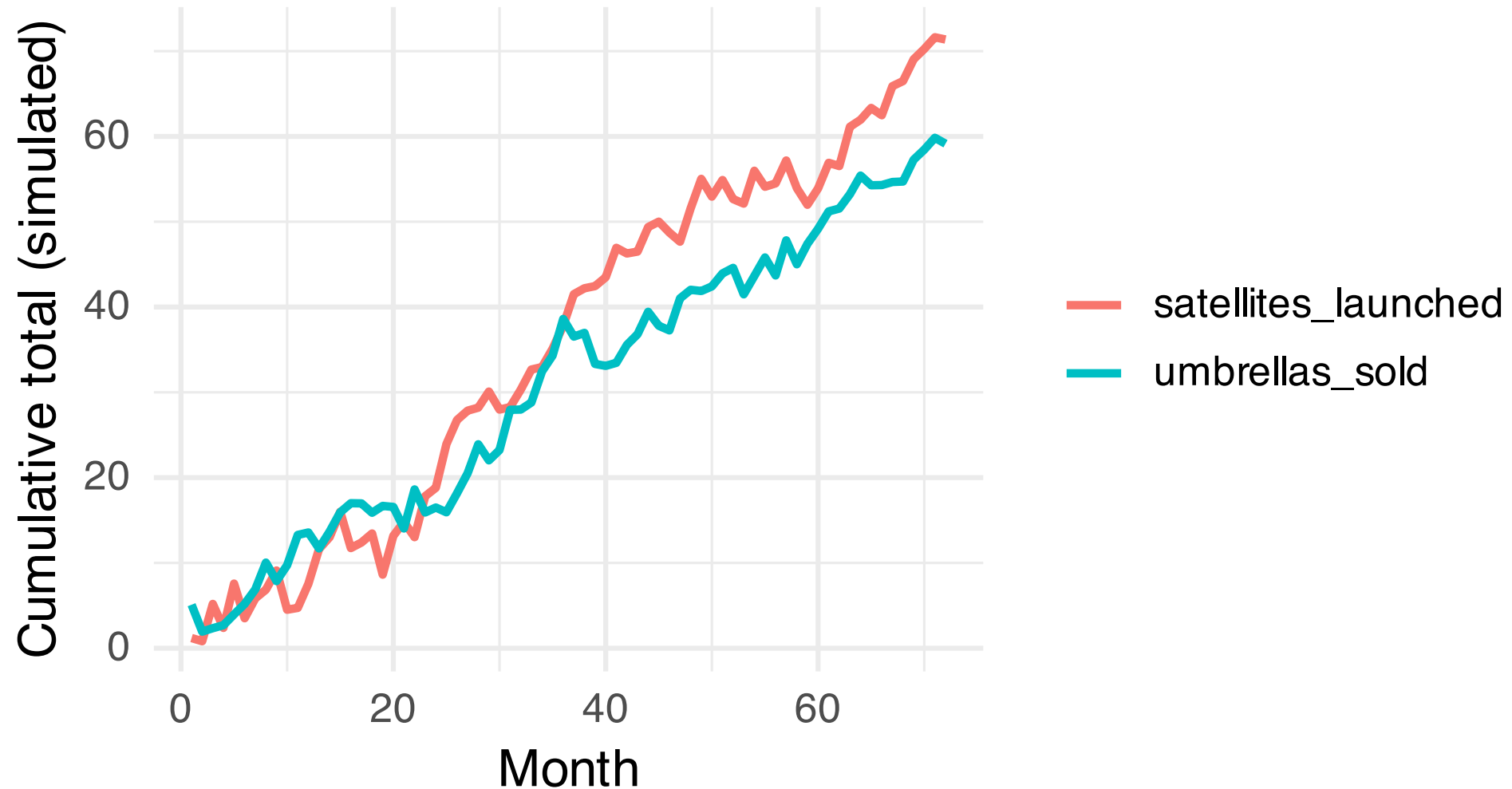
```
# A tibble: 1 × 1  
  cor  
  <dbl>  
1 0.983
```

- `umbrellas_sold` and `satellites_launched` are two independently simulated random walks — noise added up over time, with no shared cause and no connection to each other
- They correlate at 0.98 anyway, purely because both happen to drift upward over this stretch

Seeing the “Shared Trend”

```
trend_data %>%  
  pivot_longer(-month, names_to = "series", values_to = "value") %>%  
  ggplot(aes(x = month, y = value, color = series)) +  
  geom_line(linewidth = 1) +  
  labs(x = "Month", y = "Cumulative total (simulated)", color = NULL,  
        title = "Two Unrelated Series, Drifting Together by Chance") +  
  theme_minimal(base_size = 13)
```

Two Unrelated Series, Drifting Together by Chance



- Both lines drift upward together — not because one causes the other, but because random noise, added up over time, tends to wander in one direction for a while
- This is the mechanism behind nearly every “two things trending up together” chart you’ll see online

Takeaway: a high correlation coefficient, by itself, is not evidence of a relationship. Checking for a plausible mechanism — and checking whether both variables are simply trending with time — catches most spurious correlations.

Breakout: Finding (and Breaking) a Spurious Correlation

Using the same setup as `trend_data`:

1. Re-run the simulation with a `set.seed()` of your choice. What correlation do you get?
2. Try at least three different seeds. How much does the correlation bounce around?
3. Now set `mean = 0` in **both** `rnorm()` calls (no drift in either direction) and re-run. What happens to the correlation, and why?

Open `lecture11_inclass_exercise.R` and work through it there.

04

Confounding Variables

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables**
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap

Confounding Variables

A **confounding variable** Z influences both X and Y . X and Y end up correlated with each other — not because either one causes the other, but because both are responding to Z .

Classic illustration: ice cream sales and drowning deaths rise and fall together over the course of a year. Neither causes the other — both are driven by a third variable, **temperature**. On hot days, more people buy ice cream *and* more people go swimming, which raises drowning risk.

Unlike a spurious correlation, a confounded relationship has a real, identifiable cause behind it — it's just not the cause the raw correlation suggests.

Confounding, Built From Scratch

A constructed illustration: `temperature_f` drives both `ice_cream_sales` and `drownings` directly. There is no link between the two of them at all.

```
set.seed(257)
n <- 240

heat_data <- tibble(
  temperature_f = rnorm(n, mean = 72, sd = 15),
  ice_cream_sales = 300 + 10 * temperature_f + rnorm(n, 0, 90),
  drownings = 1 + 0.1 * temperature_f + rnorm(n, 0, 0.9)
) %>%
  mutate(ice_cream_sales = pmax(round(ice_cream_sales), 0),
         drownings = pmax(round(drownings), 0))

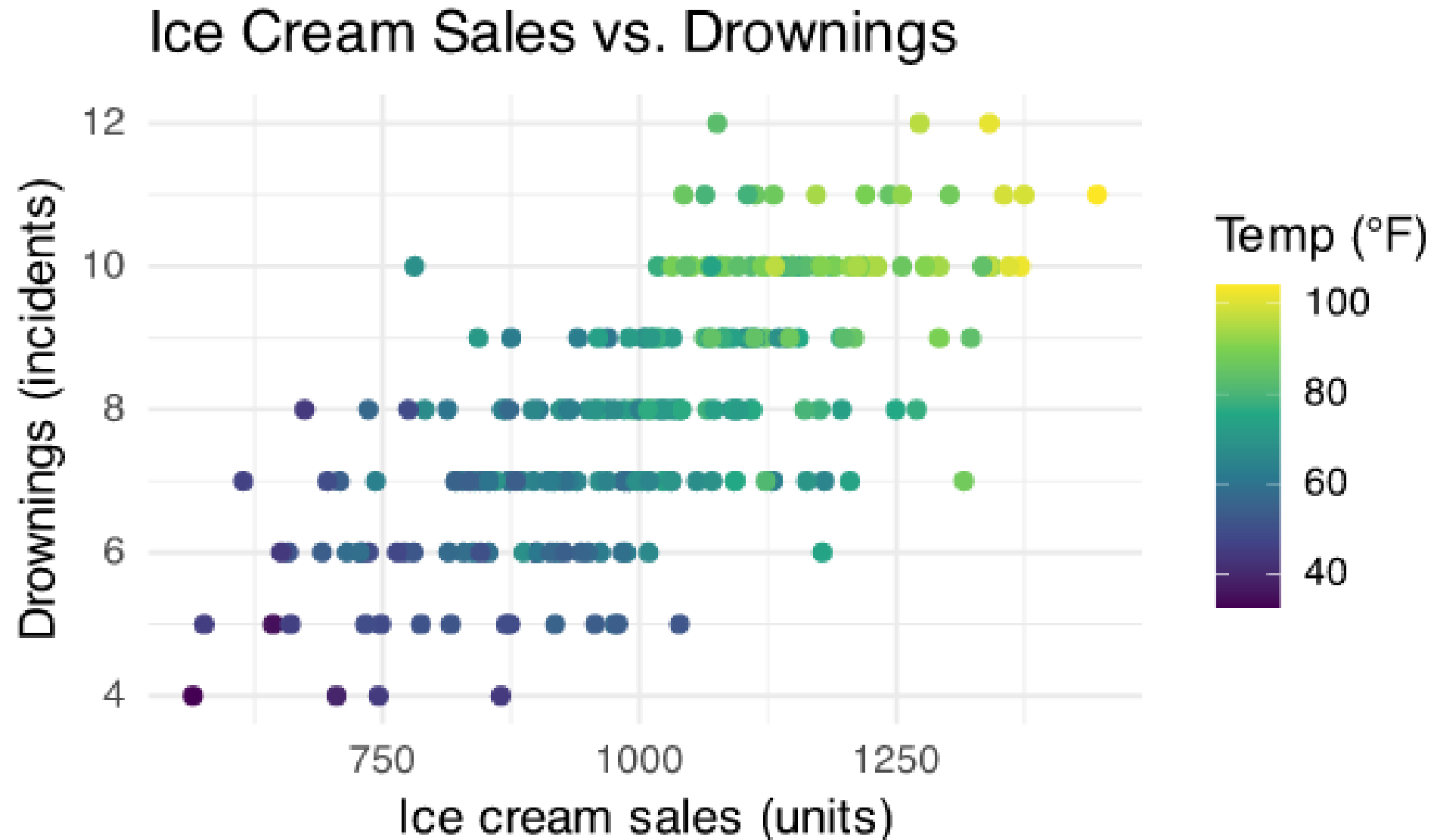
heat_data %>%
  summarise(cor = cor(ice_cream_sales, drownings))
```

```
# A tibble: 1 × 1
  cor
<dbl>
1 0.709
```

- By construction, `ice_cream_sales` and `drownings` only depend on `temperature_f` — yet they come out correlated at 0.71 with each other

Making the Real Driver Visible

```
heat_data %>%  
  ggplot(aes(x = ice_cream_sales, y = drownings, color = temperature_f)) +  
  geom_point() +  
  scale_color_viridis_c() +  
  labs(x = "Ice cream sales (units)", y = "Drownings (incidents)",  
        color = "Temp (°F)", title = "Ice Cream Sales vs. Drownings") +  
  theme_minimal(base_size = 13)
```



- Color by temperature, and the real driver becomes visible: cool days cluster low on both axes, hot days cluster high on both

Controlling for the Confounder

Split the data into temperature bands with `ntile()`, and recompute the correlation *within* each band:

```
heat_data %>%  
  mutate(temp_band = ntile(temperature_f, 3),  
         temp_band = case_when(  
           temp_band == 1 ~ "Cool days",  
           temp_band == 2 ~ "Mild days",  
           temp_band == 3 ~ "Hot days"  
         ),  
         temp_band = factor(temp_band, levels = c("Cool days", "Mild days", "Hot  
days"))) %>%  
  group_by(temp_band) %>%  
  summarise(cor = cor(ice_cream_sales, drownings))
```



```
# A tibble: 3 × 2
  temp_band cor
  <fct>      <dbl>
1 Cool days 0.286
2 Mild days 0.198
3 Hot days  0.265
```

- Overall: $r = 0.71$. Within any single temperature band: $r \leq 0.29$.
- Once temperature is held roughly fixed, most of the relationship between ice cream sales and drownings disappears — because temperature was doing the work all along.

Back to the Redfin Data

Recall the Redfin home-sales data from Lectures 8–9: homes bought and sold in the DC metro area, September 2017.

- Bigger homes should cost more overall — but does a bigger home really cost *less per square foot*? That's what the raw correlation between `square_feet` and `ppsf` (price per square foot) suggests.

```
redfin_main <- redfin %>%  
  filter(property_type %in% c("Condo/Co-op", "Single Family Residential", "Townhouse"))  
  
redfin_main %>%  
  summarise(cor = cor(square_feet, ppsf, use = "complete.obs"))
```

```
# A tibble: 1 × 1  
  cor
```

```
<dbl>  
1 -0.180
```

- Before trusting that number: what else might be moving with both `square_feet` and `ppsf`?

Group Means Tell the Real Story

```
redfin_main %>%  
  group_by(property_type) %>%  
  summarise(n = n(),  
            mean_sqft = mean(square_feet, na.rm = TRUE),  
            mean_ppsf = mean(ppsf, na.rm = TRUE))
```

```
# A tibble: 3 × 4  
  property_type      n mean_sqft mean_ppsf  
  <chr>          <int>    <dbl>    <dbl>  
1 Condo/Co-op      460      966.     442.  
2 Single Family Residential 336    2421.     369.  
3 Townhouse        346    1615.     409.
```

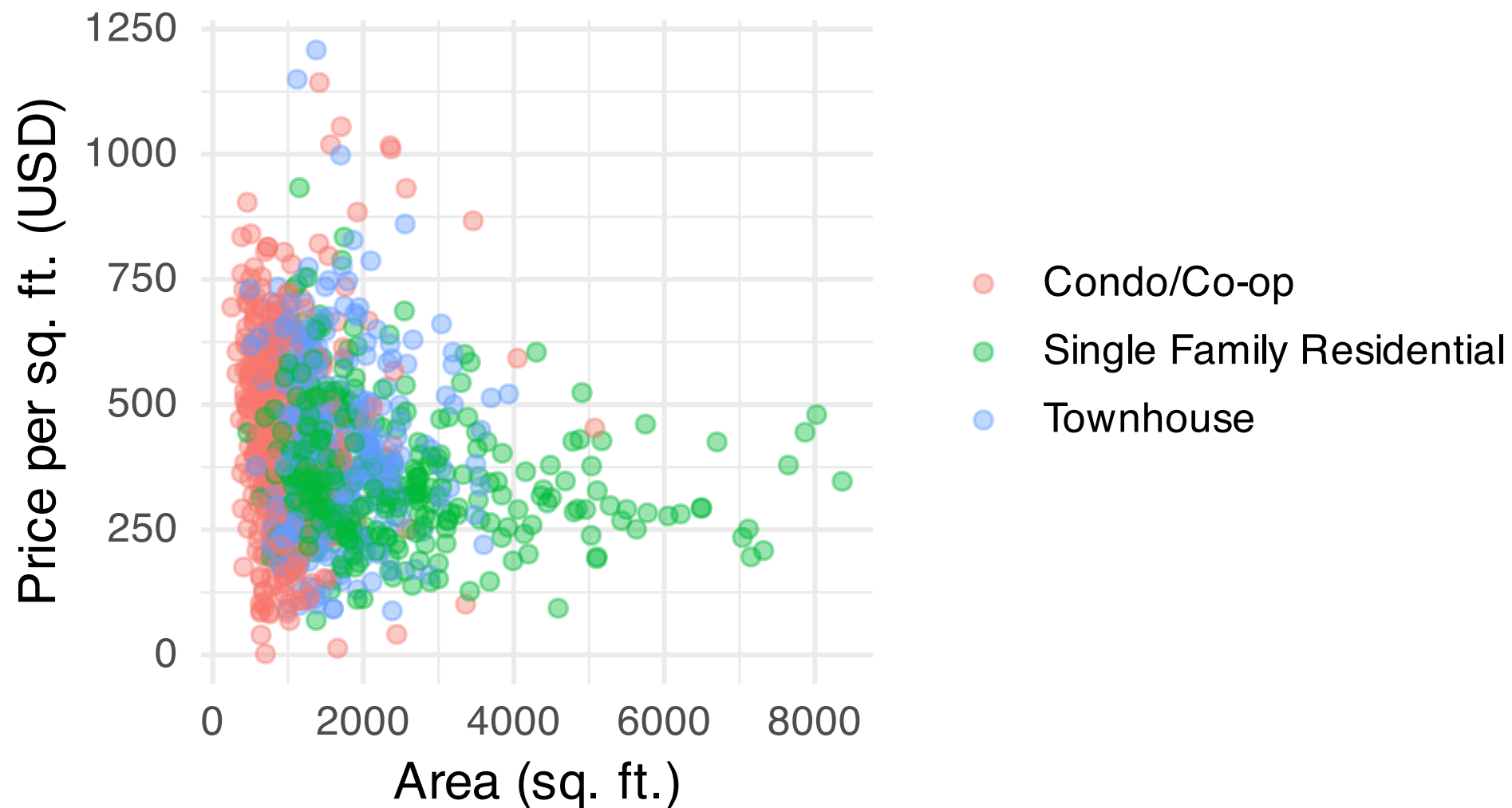
- Condos are small and expensive per square foot; single-family homes are large and cheap per square foot; townhouses sit in between

- That staircase pattern, by itself, could produce a negative overall correlation between `square_feet` and `ppsf` — even if size has nothing to do with price per square foot *within* a given property type

Visualizing the Three Clusters

```
redfin_main %>%  
  ggplot(aes(x = square_feet, y = ppsf, color = property_type)) +  
  geom_point(alpha = 0.4) +  
  labs(x = "Area (sq. ft.)", y = "Price per sq. ft. (USD)", color = NULL,  
        title = "Price per Sq Ft vs. Area, by Property Type") +  
  theme_minimal(base_size = 13)
```

Price per Sq Ft vs. Area, by Property Type



- Within each color, the downward slope is much flatter than the overall pattern you'd see collapsing all three colors together

Overall vs. Within-Group Correlation

```
redfin_main %>%  
  group_by(property_type) %>%  
  summarise(cor = cor(square_feet, ppsf, use = "complete.obs"))
```

```
# A tibble: 3 × 2  
  property_type      cor  
  <chr>          <dbl>  
1 Condo/Co-op      -0.0235  
2 Single Family Residential -0.237  
3 Townhouse        -0.0439
```

- Pooled across property types: $r = -0.18$
- Within each property type: Condo/Co-op $r \approx -0.02$, Townhouse $r \approx -0.04$, Single Family Residential $r \approx -0.24$

- Most of the pooled correlation is coming from the *differences between* property types, not from a real within-type relationship — `property_type` is confounding `square_feet` and `ppsf`.

How to check for a confounder: ask what else might move with both X and Y , then compare the raw correlation to the correlation computed separately within each level of that variable. A big drop is a red flag.

Breakout: Is `property_type` Confounding This Too?

Using `redfin_main` (the same three property types), pick a different pair of variables — for example `beds` and `price`, or `metro_distance` and `price`.

1. Compute the overall correlation.
2. Compute the correlation within each `property_type`.
3. Does the within-group correlation stay close to the overall number, or does it move a lot?
4. If it moves a lot, is `property_type` confounding that relationship too? If it barely moves, what does that tell you?

Open `lecture11_inclass_exercise.R` and work through it there.

05

Reverse Causation

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation**
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap

Reverse Causation

Sometimes X and Y are correlated because X causes Y — and sometimes it's the other way around, and the correlation coefficient alone can't tell you which.

Example: children in homes with more books tend to read better. Does owning more books cause a child to read better? Or does being a strong reader — and belonging to a family that values reading — lead a household to own more books?

The correlation is the same number either way. Figuring out the direction requires something *outside* the correlation itself: a timeline, a natural experiment, or at least a plausible mechanism.

More Examples to Reason Through

For each pair, which direction would you check first — and how would you check it?

- Cities with more police spending have less crime. Does spending reduce crime, or does more crime lead cities to spend more on policing?
- People who exercise more often report better moods. Does exercise improve mood, or are people in a better mood more likely to exercise?
- Students who work with a tutor get better grades. Does tutoring raise grades, or do students (or families) who are already more invested in grades seek out tutors?

How to check: think about timing — which happened first? Is there a mechanism that would run the other direction just as easily as the one you assumed?

06

Selection Bias

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias**
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap

Selection Bias

Selection bias is different from the first three reasons: the problem isn't a third variable, and it isn't a reversed arrow — it's that the *sample itself* isn't representative of the population you want to learn about.

- A non-representative sample can **manufacture** a relationship that doesn't exist in the full population
- A non-representative sample can also **hide** a relationship that does exist in the full population

Either way, the fix isn't a better statistical test — it's asking who ended up in your data, and who didn't.

A Constructed Illustration: Range Restriction

Simulate an applicant pool where underlying *ability* drives both SAT score and eventual college GPA:

```
set.seed(22)
n <- 2000

applicants <- tibble(
  ability = rnorm(n),
  sat_score = round(1000 + 180 * ability + rnorm(n, 0, 90)),
  college_gpa = round(3.0 + 0.35 * ability + rnorm(n, 0, 0.4), 2)
) %>%
  mutate(sat_score = pmin(pmax(sat_score, 400), 1600),
         college_gpa = pmin(pmax(college_gpa, 0), 4.3))

applicants %>%
  summarise(cor = cor(sat_score, college_gpa))
```

```
# A tibble: 1 × 1  
  cor  
  <dbl>  
1 0.605
```

- Across the full applicant pool, SAT score and college GPA are correlated at about 0.60

Restricting to Only the Admitted

A selective college only observes the students it actually admits — say, the top 10% of SAT scores in this pool:

```
admitted <- applicants %>%  
  filter(sat_score >= quantile(sat_score, 0.90))  
  
admitted %>%  
  summarise(n = n(), cor = cor(sat_score, college_gpa))
```

```
# A tibble: 1 × 2  
      n    cor  
  <int> <dbl>  
1    201 0.321
```

- Within the admitted group alone, the correlation drops to about 0.32 — roughly half of what it was in the full pool
- SAT score didn't change its relationship to GPA. The *sample* changed: once you only look at students who already cleared a high bar, there's much less variation left in SAT score to explain anything
- This is **restriction of range** — a specific, well-documented form of selection bias. It shows up whenever your sample has already been filtered on one of the variables you're studying.

Other Ways Selection Bias Shows Up

- **Survey non-response.** People willing to complete a long survey about financial habits may not look like the typical person
- **Survivorship.** Studying only firms that are still in business skips every firm the pattern you're testing might have driven out of business
- **Self-selection.** Comparing students who chose to attend office hours to those who didn't mixes "office hours helped" with "the kind of student who shows up to office hours was already different"

How to check: ask how an observation ended up in your data at all, and whether that process is related to the variables you're studying.

07

Simpson's Paradox

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox**
- 08 Checking Your Own Project
- 09 Recap

Simpson's Paradox

Simpson's paradox: a relationship that reverses direction when you split the data by a group variable, compared to when you pool everyone together.

It isn't really a paradox — it's confounding taken to its most dramatic extreme. The group variable is unevenly distributed across whatever you're comparing, and pooling hides that imbalance.

A Real Example: Kidney Stone Treatment

A 1986 study compared two treatments for kidney stones — open surgery and a less invasive procedure (percutaneous nephrolithotomy) — broken out by whether the stones were small or large.

Source: Charig et al. (1986), *British Medical Journal*; figures as summarized in the Wikipedia article “Simpson’s paradox.”

```
kidney_stones <- tribble(  
  ~treatment,      ~stone_size,      ~success, ~total,  
  "Open surgery",  "Small stones",   81,      87,  
  "Percutaneous", "Small stones",  234,     270,  
  "Open surgery",  "Large stones",  192,     263,  
  "Percutaneous", "Large stones",   55,      80  
)  
  
kidney_stones
```



```
# A tibble: 4 × 4
  treatment    stone_size success total
  <chr>        <chr>      <dbl> <dbl>
1 Open surgery Small stones      81     87
2 Percutaneous Small stones    234    270
3 Open surgery Large stones    192    263
4 Percutaneous Large stones     55     80
```

Within Each Stone Size, Open Surgery Wins

```
kidney_stones %>%  
  mutate(success_rate = percent(success / total, accuracy = 1))
```

```
# A tibble: 4 × 5  
  treatment    stone_size success total success_rate  
  <chr>        <chr>      <dbl> <dbl> <chr>  
1 Open surgery Small stones      81     87 93%  
2 Percutaneous Small stones    234    270 87%  
3 Open surgery Large stones    192    263 73%  
4 Percutaneous Large stones     55     80 69%
```

- Small stones: open surgery 93% vs. percutaneous 87%
- Large stones: open surgery 73% vs. percutaneous 69%
- Open surgery has the higher success rate in **both** groups

Pooled Across Both Groups, It Flips

```
kidney_stones %>%  
  group_by(treatment) %>%  
  summarise(success = sum(success), total = sum(total)) %>%  
  mutate(success_rate = percent(success / total, accuracy = 1))
```

```
# A tibble: 2 × 4  
  treatment      success total success_rate  
  <chr>          <dbl> <dbl> <chr>  
1 Open surgery    273   350 78%  
2 Percutaneous    289   350 83%
```

- Pooled: open surgery 78% vs. percutaneous 83%
- Percutaneous now has the **higher** overall success rate — the opposite of what both subgroups showed

Why: An Uneven Case Mix

```
kidney_stones %>%  
  group_by(treatment) %>%  
  mutate(share = percent(total / sum(total), accuracy = 1)) %>%  
  select(treatment, stone_size, share)
```

```
# A tibble: 4 × 3  
# Groups:   treatment [2]  
  treatment      stone_size  share  
  <chr>         <chr>      <chr>  
1 Open surgery Small stones 25%  
2 Percutaneous Small stones 77%  
3 Open surgery Large stones 75%  
4 Percutaneous Large stones 23%
```

- Open surgery was used on large (harder) stones 75% of the time

- Percutaneous was used on large stones only 23% of the time — it was mostly used on the easier, small-stone cases
- Stone size drives both which treatment a patient got *and* the chance of success. Pooling across stone size without accounting for that imbalance makes the treatment that was worse in both groups look better overall.

How to check: before pooling across a group variable, check whether that variable is distributed evenly across whatever you're comparing. If it isn't, compute the comparison within each group before you trust the pooled number.

Breakout: Stress-Testing the Kidney Stone Example

Using `kidney_stones` from a few slides ago:

1. Recompute the success rate within each stone size, and the pooled success rate across both — by hand first, then check your work in R.
2. Suppose percutaneous had instead been used on large stones *just as often* as open surgery, instead of only 23% of the time. Would you still expect a reversal when you pool? Why or why not?
3. In your own words: what has to be true about how patients were assigned to treatment for Simpson's paradox to show up here?

Open `lecture11_inclass_exercise.R` and work through Question 1 there.

08

Checking Your Own Project

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project**
- 09 Recap

Checking Your Own Project

Today's lecture lands the same day your first full project draft is due. Before you call a relationship in your data “interesting,” run it through the five ideas from today.

Five Questions to Ask About Your Own Analysis

1. **Confounding** — is there an obvious third variable that could be driving both sides of the relationship you're reporting?
2. **Reverse causation** — could the outcome you're explaining actually be causing your explanatory variable, instead of the other way around?
3. **Selection bias** — is your sample restricted in a way (who responded, who was included, who was excluded) that could create or hide the pattern you found?
4. **Spurious correlation** — if both variables are measured over time, could the “relationship” just be two things trending in the same direction?
5. **Simpson's paradox** — does splitting your data by an important group variable (location, property type, demographic group, time period) change the story, or even reverse it?

None of this means you need a randomized experiment to write a good project. It means naming the limitation and discussing it honestly — that's what a careful applied analysis looks like, and it's exactly what your project draft should show.

09

Recap

- 01 Recap & Agenda
- 02 Why Correlation Isn't Causation
- 03 Spurious Correlation
- 04 Confounding Variables
- 05 Reverse Causation
- 06 Selection Bias
- 07 Simpson's Paradox
- 08 Checking Your Own Project
- 09 Recap**

Recap

- **Spurious correlation** — chance or a shared trend, no real connection between the variables at all
- **Confounding** — a third variable drives both X and Y ; compare the pooled correlation to the correlation computed within each level of that variable
- **Reverse causation** — the correlation can't tell you which variable came first; think about timing and mechanism
- **Selection bias** — a non-representative sample can create or hide a relationship; ask how an observation ended up in your data
- **Simpson's paradox** — a relationship that reverses once you split by a group variable, usually because that group isn't evenly distributed across what you're comparing

Functions Introduced Today

- `group_by() + summarise(cor = cor(...))` — correlation computed separately within each level of a grouping variable
- `ntile()` — splitting a continuous variable into evenly-sized bands for the same kind of group comparison
- `tribble()` — building a small data frame by hand, row by row
- `scales::percent()` — formatting a proportion as a percentage for display

Documentation & Further Reading

- `?cor`, `?group_by`, `?summarise` — correlation within groups
- `?dplyr::ntile` — splitting a variable into evenly-sized groups
- <https://www.tylervigen.com/spurious-correlations> — the “Spurious Correlations” project
- https://en.wikipedia.org/wiki/Simpson%27s_paradox — the kidney stone example, with sourcing
- <https://en.wikipedia.org/wiki/Confounding> — confounding variables in more depth

Coming up: project presentations.